

Hospital Queue Management system

Table Of Contents

	<u>Pages</u>
Introduction	3
Problem Statement.....	4
Objectives.....	5
Data Structure Justification.....	6
System Design.....	9
Algorithms.....	17
Implementation Details.....	24
Sample Input/Output.....	33
Conclusion.....	39

1.Introduction

- Healthcare systems require efficient and reliable methods to manage patient flow, especially in emergency situations where time and prioritization are the most critical terms. In traditional queue systems, patients are treated on a first-come, first-served basis. However, this approach is not suitable in medical environments, where patients with severe conditions must receive immediate attention regardless of their arrival time.
- This project presents a Hospital Queue Management System designed to prioritize patients based on the severity of their medical condition. The system is implemented using the concept of a priority queue, which allows patients with higher urgency to be treated before others which have mild status.
- The system is developed using C++, applying efficient data structures. It supports essential operations such as adding new patients, serving patients, searching for patient records, updating patient information, doctoroverride and maintaining persistent data through file handling functionalities.
- By simulating a real-world hospital scenario, this system demonstrates how data structures and algorithms can be applied to solve practical problems efficiently. The project emphasizes performance, reliability, and usability, making it a valuable tool for understanding both theoretical and practical aspects of software engineering.

2.Problem Statement

- ❖ In hospital environments, patient management is a critical task that directly affects the quality of healthcare services. Traditional queue systems, which operate on a **First-In-First-Out (FIFO)** basis, are not suitable for medical settings because they do not consider the severity of patients' medical conditions. As a result, patients with life-threatening conditions may experience dangerous delays which could have fatal results if they arrive after patients with minor illnesses.
- ❖ The main problem addressed in this project is the need for an efficient system that can **prioritize patients based on medical urgency** while maintaining fairness among patients with similar conditions, which the system will handle them in terms of arrival time. The system must be capable of handling multiple operations, including patient registration, treatment processing, searching for patient records, updating patient information, allowing doctor override and maintaining persistent storage of data.

Additionally, the system should ensure:

- Allowing only authorised users to have a role in the system by forcing login actions and those roles will work on their corresponding fields(ie. Working only on allowed functionalities).
 - Accurate and fast retrieval of high-priority patients(serving based on priority)
 - Prevention of duplicate patient records (by using patient ID as an identifier)
 - Ease of use through proper input validation(validating inputs and empty inputs)
 - Reliability through data storage and retrieval mechanisms (using file system)
- ★ Therefore, our goal is to design and implement a system that efficiently manages patient queues using an appropriate data structure, ensuring that critical patients receive immediate attention without compromising system performance or usability.

3.Objectives

- ❖ The main objective of this project is to design and implement an efficient Hospital Queue Management System that prioritizes patients based on the severity of their medical conditions using an appropriate data structure (ie.**priority queue**).

Specific Objectives

- To develop a system that **prioritizes patients** according to the urgency of their medical condition rather than their arrival time. But also serves based on arrival time when patients have similar statuses(priority).
- To develop a system that allows only users which are authorised.
- To implement a **priority queue data structure** for the goal of efficient management of patient records and treatment order.
- To provide essential functionalities including:
 - Log in options
 - Adding new patients
 - Serving patients
 - Displaying all patients in the queue
 - Searching for a specific patient
 - Updating patient information
 - Doctor override option
 - Cancelling appointment
- To ensure **data integrity** by preventing duplicate patient IDs and validating user inputs.
- To incorporate **file handling mechanisms** that allow saving and loading patient data for persistence.
- To design a system that is **user-friendly**, reliable, and easy to operate through a menu-driven interface.

- To demonstrate the practical application of **data structures and algorithms** in solving real-world problems.

4.Data Structure Justification

- ❖ Selecting an appropriate data structure is critical for ensuring the efficiency and correctness of any software system. In this project, a **Priority Queue** has been chosen as the core logical data structure to manage patient records.
- ❖ The system is implemented using a **priority queue which is built based on a singly linked list**. Each node in the linked list stores a patient's information along with a pointer to the next node. Unlike a simple queue, this implementation ensures that patients are not processed strictly in arrival order but based on the severity of their condition yet it also gives attention to arrival time.
- ❖ The priority queue is organized using two criteria: **priority level** and **arrival time**. Patients with more critical conditions are assigned higher priority (lower numerical value), and in cases where multiple patients share the same priority, the system uses arrival time to determine the order of service. This ensures fairness while still prioritizing urgent medical cases.
- ❖ This approach allows dynamic memory allocation and efficient insertion of patients at the correct position without needing to rearrange the entire structure, making it suitable for real-time hospital queue management.

Why Priority Queue is Suitable

- ❖ In a hospital environment, patients must be treated based on the **severity of their condition**, not simply their arrival time. A standard queue (FIFO) would process patients in the order they arrive, which could delay treatment for patients in critical statuses. Therefore, a more advanced data structure is required to handle this priority-based processing.
- ❖ A **priority queue** allows each patient to be assigned a priority level, ensuring that patients with more urgent conditions are served before those with less severe (mild) conditions.

Priority Criteria Used in the System

The system defines patient priority as follows:

- **Priority 1:** Severe Injury (Highest priority)
- **Priority 2:** Serious Illness
- **Priority 3:** Mild Condition (Lowest priority)

- In cases where two patients have the same priority level, the system uses **arrival time** as a secondary criterion. This ensures fairness by treating patients with equal severity in the order of their arrival time. This feature indirectly adds the FIFO structure .

Implementation Approach

1.The priority queue:

- ❖ The system uses a **custom priority queue implemented using a singly linked list** instead of relying on the C++ Standard Template Library.

Reason for Using a Custom Implementation

- ❖ Although C++ provides a built-in `priority_queue`, this project implements a custom priority queue to:
 - Gain deeper understanding of **data structures and memory management**
 - Demonstrate manual control over **insertion and deletion operations (enqueue&dequeue)**
 - Improve the knowledge of **pointer-based data structures**
 - Fulfill academic requirements emphasizing **implementation from scratch**

2. Major Functionalities used:

- ➔ **Add patient():**Used to add patient (enqueue) records into the patients queue.
- ➔ **Serve patient():**Used to serve patients based on their priority (dequeue).
- ➔ **Display patients():**Used to Display patients records in the queue.
- ➔ **Load and save ():**Used to load records from the file(during the case of extended schedules) and to save records into a file respectively.
- ➔ **Update():**Used to update patient records.
- ➔ **Cancel():**Used to cancel patient appointments by using corresponding ID.
- ➔ **Doctoroverride():**Used to modify patients priority(condition which is entered by the system) based on the decision of the doctors
- ➔ **Login():**Used to allow only authorised users (ie, Receptionist and Doctor only).
- ➔ **Queue functions():** This includes functions such as **enqueue(),dequeue()**and **isempty()** functions which are used for implementing the basic functions of priority queue data structure.
- ➔ **Helper():**Used for input validation.

3. Main Function: The main function consists of a do while loop which contains switch cases for handling function calls, login/logout actions and system termination during the working of the system.

Efficiency Analysis

The Time efficiency analysis of the system can be described based on the frequently used functionalities:

- **Insertion (Add Patient)(Enqueue with traversing):** $O(n)$
 - **Deletion (Serve Patient):** $O(n)$
 - **Access Highest Priority Patient(Dequeue):** $O(1)$
- These time complexities make the system efficient even when handling a large number of patients.

Over all analysis

Function Group	Function Name	Best-Case	Worst-Case
Core Queue	enqueue()	$O(1)$	$O(n)$
	dequeue()	$O(1)$	$O(1)$
Search&Display	Display_sPatients()	$O(1)$	$O(n)$
	Display_Patient()	$O(1)$	$O(n)$
Modification	cancelpatient()	$O(1)$	$O(n)$
	updatePatient()	$O(1)$	$O(n)$
	doctorOverride()	$O(1)$	$O(n)$

File/o	SaveToFile()	O(n)	O(n)
	LoadFromFile()	O(1)	O(n ²)
Helper	getValidint()	O(1)	O(1)
	getValidInput()	O(1)	O(1)

Advantages of Using Customized Priority Queue

- ❖ Ensures **correct treatment order** based on urgency
 - ❖ Provides **efficient operations** for dynamic data
 - ❖ Maintains **automatic sorting**, reducing manual effort
 - ❖ Supports **real-time decision making**
 - ❖ Full control over data structure behavior
 - ❖ Better understanding of linked list operations
 - ❖ Flexible and customizable logic Limitations
 - ❖ Direct searching is not efficient and requires traversal
 - ❖ Slower insertion compared to heap-based implementation
 - ❖ Requires manual memory management
 - ❖ More complex to implement
- Despite these limitations, the priority queue remains the most suitable choice for this application due to its ability to handle **priority-based processing effectively**.

5. System Design

- ❖ The Hospital Queue Management System is designed as a **menu-driven application** that allows users to interact with the system through a series of options. The system follows a modular approach, where each functionality is implemented as a separate function to ensure clarity, maintainability, and reusability.

Overall System Architecture

The system consists of three main components: →

User Interface Layer

- Handles interaction with the user through console input/output

- Displays menus and receives user input
- Shows results and confirmation and denial messages

→ Processing Layer

- Contains the core logic of the system
- Handles operations such as login/logout, adding, serving, searching, and updating patients
- Controls the user role restrictions → Data

Management Layer

- Manages the priority queue data structure
- Handles file storage and retrieval
- Controls data integrity and availability

Data Representation

Each patient is represented using a structured data type:

- **Patient ID** – Unique identifier
 - **Name** – Patient's name
 - **Condition** – Medical condition description
 - **Priority** – Numeric value representing urgency
 - **Arrival Time** – Used to maintain fairness among equal priorities
- This structured representation ensures that all relevant information is stored and processed efficiently.

Priority Handling Mechanism

- ❖ The core operations of the system are based on a priority queue mechanism.
- ❖ The **enqueue operation** inserts a new patient into the queue at the appropriate position based on their priority and arrival time. If the queue is empty or the new patient has the highest priority, they are inserted at the front. Otherwise, the system traverses the list to find the correct position.
- ❖ The **dequeue operation** removes the patient at the front of the queue, which represents the patient with the highest priority. This operation ensures that urgent cases are always handled first.
- ❖ Additional operations such as searching, updating, and deletion are performed by traversing the linked list and modifying node connections accordingly.

- **This ensures both correct prioritization and fairness.**

Actors and Roles

The Hospital Queue Management System is designed using a **role-based interaction model** to simulate how different personnel operate within a real hospital environment. Instead of allowing unrestricted access to all system functionalities, the system separates responsibilities among different actors to ensure better organization, control, and realism.

This approach reflects real-world hospital workflows, where different staff members are responsible for specific tasks and decisions.

Identified Actors

The system includes two primary actors:

1. Receptionist

- ❖ The receptionist acts as the **main system operator** and is responsible for handling all patient-related administrative tasks. This role interacts with the system more frequently than any other and is essential for maintaining the queue.

Detailed Responsibilities:

- **Patient Registration:**
The receptionist collects patient information such as ID, name, and medical condition. Based on the condition selected, the system automatically assigns a priority level.
- **Queue Management:**
Once a patient is registered, the receptionist adds them to the queue. The system ensures proper placement using priority-based insertion.
- **Serving Patients:**
The receptionist processes patients in order of priority by removing them from the queue when they are ready to be treated.
- **Updating Patient Information:**
If a patient's condition or details change, the receptionist can modify the record. The system then repositions the patient in the queue if necessary.
- **Cancelling Appointments:**
Patients who withdraw or are no longer available can be removed from the queue.
- **Data Persistence:**

The receptionist is responsible for saving and loading patient data to ensure continuity between system sessions.

Role Importance:

- The receptionist ensures that the system remains **organized, accurate, and up-to-date**, making this role critical for the smooth functioning of the entire system.

2. Doctor

- ❖ The doctor has a **specialized and limited role**, focused primarily on handling exceptional or emergency cases.

Detailed Responsibilities:

- **Priority Override:**
The doctor can manually adjust the priority level of any patient in the queue. This is especially important in cases where a patient's condition becomes more severe after registration.
- **Emergency Handling:**
By overriding priority, the doctor ensures that urgent cases are handled immediately, regardless of their original queue position.

Role Importance:

Although the doctor interacts less frequently with the system, their role is **critical for patient safety and responsiveness**, as it allows real-time adjustments based on medical judgment.

Role-Based Access Mechanism

- ❖ The system implements a **basic login mechanism** to control access to functionalities based on user roles.
- Users must log in using predefined credentials
- Once authenticated, the system assigns a role:
 - Receptionist
 - Doctor
- Each function in the system checks the current role before execution. If a user attempts to access a restricted function, the system blocks the action and displays an appropriate message.

Examples:

- A doctor cannot add or remove patients

- A receptionist cannot perform priority overrides

This ensures that users only perform actions relevant to their responsibilities.

Important Implementation Note

- ❖ It is important to clarify that the role-based system in this project is **conceptual and logical rather than fully secure**.
- The login credentials are **hardcoded within the program**
- There is **no database, encryption, or authentication server**
- Access control is enforced using simple conditional checks (**if** statements)
- Therefore, these roles should be understood as **system-level controllers used for simulation purposes**, rather than real-world authenticated users.

Functional Modules

- ❖ The Hospital Queue Management System provides a comprehensive set of operations to manage patient flow efficiently.
- ❖ The system allows users to **add patients** with validated inputs, ensuring that each patient has a unique ID and properly formatted name. Once added, patients are inserted into the queue based on their priority and arrival time.
- ❖ The system also supports **serving patients**, where the highest-priority patient is removed from the queue and displayed. Users can **view all patients** currently in the queue in their correct order, as well as **search for a specific patient** using their unique ID.
- ❖ Advanced functionalities include the ability to **update patient information**, which involves removing the patient from the queue, modifying their details, and reinserting them in the correct position. The system also allows **cancellation of patients**, effectively removing them from the queue.
- ❖ A special feature called **doctor override** enables manual adjustment of a patient's priority, ensuring that emergency decisions can be handled flexibly. The system further supports **saving and loading data from a file**, allowing persistent storage of patient records.

The system is divided into the following functional modules:

1.Login function

- Allows users to get into the system by authorising them using user name and password.

2. Add Patient

- Accepts patient details
- Assigns priority based on condition
- Inserts patient into the priority queue

3. Serve Patient

- Removes the highest-priority patient from the queue
- Displays patient details

4. Display Patients

- Shows all patients currently in the queue
- Uses a temporary queue to preserve original data

5. Search Patient

- Searches for a patient using their ID
- Displays details if found

6. Update Patient

- Modifies patient information
- Rebuilds the queue to maintain priority order

7. Save to File

- Stores patient data in a text file
- Ensures data persistence

8. Load from File

- Reads patient data from file
- Reconstructs the priority queue

9. Cancel patient

- Cancels patient from the queue
- Maintains the remaining data

10. Doctor override

- Modifies the current condition of a patient based on the doctor's decision
- Maintains the remaining data

System Flow

1. System Initialization

When the program starts:

- It automatically calls the file-loading function
- Patient data is read from `Patients.txt`
- Each record is reconstructed and inserted into the queue During this process:
- Duplicate IDs are avoided
- Arrival time is restored
- Queue order is preserved

This ensures that previously saved data is not lost between sessions.

2. Authentication Process

Before accessing the system:

- The user must log in
- The system validates credentials

Depending on the input:

- If `"admin"` → Receptionist role
- If `"doctor"` → Doctor role

If invalid credentials are entered:

- Access is denied
- The user is prompted to try again

This step ensures controlled system usage.

3. Patient Registration and Insertion

When adding a patient:

1. The system collects validated input:
 - ID (must be unique)
 - Name (letters only)
 - Condition (selected from menu)
2. The system assigns:
 - Priority (1, 2, or 3) &
 - Arrival time (auto-incremented)
3. The patient is inserted into the queue using:
 - Priority comparison &
 - Arrival time as a tie-breaker

Key Behavior:

- Higher priority patients are placed high on the list
- Patients with equal priority are served in arrival order

4. Queue Management Operations

The system allows several operations:

Display All Patients

- Traverses the linked list
- Displays patient details in order

Search Patient

- Searches by ID
- Displays specific patient details if found

Update Patient

- Removes the existing node
- Updates information
- Reinserts the patient in correct position

Cancel Patient

- Removes the patient node from the queue
- Updates links accordingly

5. Serving Patients

Serving a patient involves:

- Removing the first node in the queue
- This node always represents the highest-priority patient After removal:
- The queue automatically shifts
- The next patient becomes the front

This operation reflects real hospital service flow.

6. Doctor Priority Override

This is a specialized operation:

- The doctor selects a patient by ID
- Assigns a new priority The system then:
 1. Removes the patient from the current position
 2. Updates the priority
 3. Reinserts the patient based on new priority

This ensures the queue remains correctly ordered after changes.

7. File Handling and Persistence

The system uses file operations to maintain data:

Saving Data

- All patient records are written to **Patients.txt**
- Each record is stored in a structured format

Loading Data

- Reads file line by line
- Reconstructs patient objects
- Inserts into queue This allows:
- Data persistence
- Recovery after program restart

8. Logout and System Exit

- Users can log out to switch roles
- On exit, the system prompts to save data This prevents accidental data loss.

Design Considerations

- **Modularity:** Each function performs a specific task
 - **Efficiency:** Priority queue ensures fast processing
 - **Usability:** Menu-driven interface simplifies interaction
 - **Data Integrity:** Input validation and duplicate ID checks
 - **Persistence:** File handling ensures data is not lost.
-

6.Algorithms

This section describes the step-by-step procedures used to implement the main functionalities of the Hospital Queue Management System .

Algorithm: Enqueue

1. Create a new node with patient data
2. If queue is empty OR new patient has higher priority than head:
 - Insert at beginning
3. Else:
 - Traverse the list
 - Find correct position based on priority and arrival time

- Insert node in sorted position

Algorithm: Dequeue

1. Check if queue is empty
2. Store data of head node
3. Move head pointer to next node
4. Delete old head node
5. Return patient data

Algorithm: Add Patient

1. Initialize a New Patient record
 - Create a new temporary Patient object/structure.
2. Input and Validate Patient ID
 - Prompt the user to enter a Patient ID.
 - Check for Duplicates: Search the existing queue; if the ID already exists, display an error and terminate the process.
3. Input Patient Details
 - Prompt for the Patient Name (ensure it contains only letters and spaces).
 - Prompt for the Condition Choice (1. Severe, 2. Serious, 3. Mild).
 - Assign the corresponding Priority level (1 for Severe, 2 for Serious, 3 for Mild).
4. Assign Arrival Time
 - Increment the global patientcounter.
 - Assign this value to the new patient's arrival time to track their place in line relative to others with the same priority.
5. Create a New Node
 - Allocate memory for a new Node and store the Patient data within it.
6. Find the Insertion Point (Priority Enqueue)
 - Case A: Queue is Empty OR New Patient has higher priority than the Head:
 - Point the new node's next pointer to the current head.
 - Set the queue head to the new node.
 - Case B: Traverse to find position:
 - Create a temp pointer starting at the head.

- While the next patient in line has a higher or equal priority (or arrived earlier), move temp forward.
- Insert the new node after temp and before temp->next.

7. Finalize and Save

- Display "Patient added successfully."
- Call SaveToFile() to ensure the data is written to the permanent storage (Patients.txt).

Worst-case analysis Time = $O(n)$

- Because it needs to compare the priority of the existing elements of the list

Algorithm: Serve Patient

1. Check if the queue is empty
 - If hospitalQueue is NULL, display "No patients in queue" and terminate.
2. Access the highest priority patient
 - Identify the patient at the head of the linked list (this is the person with the highest priority and earliest arrival time).
3. Display patient information
 - Output the ID, Name, Condition, and Priority of the patient being served.
4. Remove the patient from the queue
 - Create a temporary pointer temp to point to the current head node.
 - Update the head pointer to point to the next node in the sequence (hospitalQueue = hospitalQueue->next).
5. Deallocate memory
 - Delete the node pointed to by temp to free up system memory.
6. Update permanent records
 - Call SaveToFile() to ensure the Patients.txt file reflects that this patient has been processed and removed from the queue.
7. Finalize
 - Display a completion message if necessary.

Worst-case analysis Time = $O(1)$

- Because it only need to dequeue the first element in the list at a time

Algorithm: Display All Patients

1. Check if the queue is empty
 - If the head pointer (hospitalQueue) is NULL:
 - Display "Queue is empty."
 - Terminate the function.
2. Initialize a traversal pointer
 - Create a temporary pointer (e.g., temp) and set it to the head of the queue.
3. Display Header
 - Print a header such as "Patients in Queue:" to format the output.
4. Traverse the linked list
 - While the temp pointer is not NULL:
 - Access Data: Print the current patient's ID, Name, Condition, and Priority level.
 - Move Next: Update the temp pointer to point to the next node in the list (temp = temp->next).
5. Finalize
 - Once the pointer reaches NULL (the end of the list), the loop terminates and the function ends.

Worst-case analysis Time = $O(n)$

- Because to print the list it must visit every single node

Algorithm: Search Patient

1. Check if the queue is empty
 - If empty, display a message and terminate
2. Prompt the user to enter Patient ID 3. Copy the queue into a temporary queue While the temporary queue is not empty:
 - Retrieve the top patient If ID matches:
 - ➔ Display patient details

If not: Traverse through the list

If id is not in the list : says "ip not found" and terminates Worst-case analysis Time = $O(n)$

- Because to print the list it must visit every single node(n) exactly ones

Algorithm: Update Patient

1. Check if the queue is empty

- If empty, display "Queue is empty" and terminate.
- 2. Prompt the user to enter Patient ID to be updated.
- 3. Initialize search pointers
 - Set the current pointer to the head of the queue. ➤
 - Set the previous pointer to NULL.
- 4. Find the patient
 - While the current patient is not NULL and the ID does not match:
 - Move previous to the current position.
 - Move current to the next patient in the queue.
- 5. If the patient is not found (current is NULL):
 - Display "Patient not found" and terminate.
- 6. Display current details (Name, Condition, Priority) of the patient found.
- 7. Prompt user for new details:
 - Enter New Name.
 - Choose New Condition/Priority (Severe, Serious, or Mild).
- 8. Remove the old record from the queue:
 - If the patient is at the front (previous is NULL):
 - Update the queue head to point to the next patient.
 - Else (patient is in the middle or end):
 - Update the previous patient's next pointer to skip the current patient.
- 9. Re-insert the patient (Maintain Priority):
 - Call the enqueue function with the updated patient data.
 - Note: This ensures the patient moves to the correct spot if their priority changed.
- 10. Deallocate memory
 - Delete the old current patient record.
- 11. Finalize:
 - Display "Patient updated successfully."
 - Call SaveToFile() to update the external record. Worst-case analysis Time = $O(n)$
 - Because it must traverse through the list in order to find and update records

Algorithm: Save to File

- 1. Check if the queue is empty
 - Ask for confirmation before overwriting

2. Open file in write mode
3. Copy queue into a temporary queue
4. While temporary queue is not empty:
 - Write patient details to file
5. Close file
6. Display success message

Worst-case analysis Time = $O(n)$

- Because it traverse the entire linked list from the head to the end

Algorithm: Load from File

1. Clear the existing queue
2. Open file in read mode
3. If file cannot be opened:
 - Display error and terminate
4. Read file line by line
5. Parse each line into patient attributes
6. Create a patient object
7. Update arrival time counter
8. Insert into priority queue
9. Update arrival time counter
10. Checks for duplicate id
11. Close file
12. Display success message

Worst-case analysis Time = $O(n)$

- Because it must read every line or record in the file

Algorithm: Cancel Patient

1. Check if the queue is empty
 - If empty, display "Queue is empty" and terminate.
2. Prompt the user to enter Patient ID
3. Initialize search pointers

- Set the current pointer to the head of the queue. ➤
Set the previous pointer to NULL.
- 4. While the current patient is not NULL and the ID does not match:
 - Move previous to the current position.
 - Move current to the next patient in the queue.
- 5. If the patient is not found (current is NULL):
 - Display "Patient not found" and terminate.
- 6. Remove the patient from the queue:
 - if the patient is at the front (previous is NULL):
 - Update the queue head to point to the next patient.
 - Else (patient is in the middle or end):
 - Update the previous patient's next pointer to skip the current patient.
- 7. Deallocate memory
 - Delete the current patient record.
- 8. Display success message
 - Display "Patient cancelled successfully."

Worst-case analysis Time = $O(n)$

- Because of traversing the entire list if the ID is at the end.

Algorithm: Doctor Override

1. Check if the queue is empty
 - If empty, display "No patients to override" and terminate.
2. Prompt the user to enter Patient ID
3. Initialize pointers
 - Set current to head.
 - Set previous to NULL.
4. While current is not NULL and ID does not match:
 - Move previous to current.
 - Move current to next patient.
5. If patient is not found (current is NULL):
 - Display "Patient ID not found".
6. Else (Patient found):
 - If patient is already at the front (previous is NULL):
 - Display "Patient is already at the front".

- Else (Move to front):
 - Link previous->next to current->next (Remove from middle).
 - Link current->next to the original head.
 - Set head to current (Make them the new first patient). ➤ Display "Doctor override successful".

Worst-case analysis Time = $O(n)$

- Because of traversing the entire list if the ID is at the end.

7.Implementation Details

- ❖ The Hospital Queue Management System is implemented using **C++**, utilizing a custom **priority queue implemented using a singly linked list and** key features such as a “patient” structure to handle patient records, the file handling, and modular programming. The implementation focuses on efficiency, clarity, and reliability.

Programming Language and Tools

- **Language Used:** C++ •

Main Libraries Used:

- `<iostream>` for input/output operations
 - `<fstream>` for file handling
 - `<sstream>` for parsing and validating input
 - `<cctype>` for validating alphabetic inputs
 - `<string>` for handling string inputs
- These libraries provide robust support for building a structured and efficient system.

Core Data Structure Implementation

- ❖ The priority queue is implemented manually using a **linked list structure** instead of the STL `priority_queue`.

Node Structure

Each node contains:

- A `Patient` object(Data)
- A pointer to the next node

Queue Representation

- A global pointer (`hospitalQueue`) points to the head of the list
- The head always contains the **highest-priority patient**

Insertion Logic (enqueue)

- A new node is created for each patient
- The node is inserted at the correct position based on:
 - Priority
 - Arrival time

Deletion Logic (dequeue)

- The first node (highest priority) is removed
- The head pointer is updated

Input Validation

- ❖ The system incorporates robust input validation mechanisms to ensure data accuracy and prevent errors during execution.
- ❖ For numeric inputs, the system verifies that the user provides valid integers and prevents empty or malformed entries. This is achieved using string-based input handling combined with safe conversion techniques.
- ❖ For textual inputs such as patient names, the system ensures that the input is not empty, does not consist solely of spaces, and contains only alphabetic characters and spaces. It also prevents invalid formatting such as leading spaces.
- ❖ These validation techniques enhance the reliability of the system and improve user interaction by reducing the likelihood of incorrect data entry.

These includes:

- `getValidInt()` ensures only valid integers are accepted
- `getValidInput()` prevents empty string inputs& valid string values
- `exists()` validates(checks) id duplications
- These functions enhance system reliability and prevent runtime errors caused by invalid user input.

Login and Verification

- ❖ The login and verification component of the system is responsible for controlling access to system functionalities based on predefined user roles. This mechanism ensures that only authorized users can perform specific operations, thereby maintaining logical separation of responsibilities within the system.
- ❖ The implementation follows a **simple role-based authentication approach**, where users must provide valid credentials before accessing the system.

Login Mechanism

- ❖ The system implements a login function that prompts the user to enter a **username** and **password**. These credentials are then verified against predefined values stored within the program.

Process Flow:

1. The system displays a login prompt requesting:
 - Username
 - Password
 2. The entered credentials are compared with predefined values:
 - "admin" with password "123" → Assigned **Receptionist role**
 - "doctor" with password "456" → Assigned **Doctor role**
 3. If the credentials match:
 - The system sets the **currentRole** variable accordingly
 - The **isLoggedIn** flag is set to **true**
 - Access to the main system menu is granted
 4. If the credentials do not match:
 - Access is denied
 - An error message is displayed
 - The user is prompted to try again
- **This loop continues until a valid login is provided.**

Security and Design Considerations

- ❖ It is important to note that the login system in this project is **basic and intended for demonstration purposes only**.

Limitations:

- Credentials are **hardcoded in the source code** • There is **no encryption or hashing of passwords**
 - No database is used for storing user accounts
 - No protection against multiple failed login attempts **Design**

Justification:

Despite these limitations, this approach is suitable for:

- Academic demonstration
- Understanding role-based system design
- Simulating real-world access control logic The implementation

effectively shows how:

- Users can be authenticated
- Roles can be assigned
- Access can be restricted based on permissions

Core Functionalities

1.Patient Registration (Add Patient)

Description

- This function allows the receptionist to register a new patient and insert them into the queue based on priority.

Implementation Details

- The system first verifies that the current user is a **receptionist** • It collects:
 - Patient ID (validated integer)
 - Patient name (validated string) ○

Medical condition (menu selection) • The system ensures:

- The ID is **unique** (checked using the `exists()` function)
- The name contains only valid characters ●

Based on the selected condition:

- Priority is assigned:
 - Severe Injury → 1
 - Serious Illness → 2
 - Mild Condition → 3
- The system assigns an **arrival time** using an auto-increment counter
- The patient is inserted into the queue using the `enqueue()` function, which:
 - Places the patient in the correct position based on priority
 - Uses arrival time to resolve ties ●

After insertion:

- The system confirms success
- Data is saved to file

2.Serving Patients (Dequeue Operation)

Description

- This function removes and processes the patient with the highest priority.

Implementation Details

- Access is restricted to the **receptionist**
 - The system checks if the queue is empty:
 - If empty → displays a message
 - Otherwise → proceeds
 - The first node (highest priority) is removed using `dequeue()`
 - The system:
 - Displays patient details
 - Updates the queue structure ○ Frees
- memory of the removed node ● After serving:
- The updated queue is saved to file

3.Display All Patients

Description

- Displays all patients currently in the queue.

Implementation Details

- The system checks if the queue is empty • If not:
 - Traverses the linked list from head to end
 - Displays each patient's:
 - ID,Name,Condition,Priority
- The display reflects the **exact order of the queue**

4.Search for a Specific Patient

Description

- Allows the user to find a patient using their ID.

Implementation Details

- The system:
 - Prompts for a patient ID
 - Traverses the linked list • If a match is found:
 - Displays full patient details • If not:
 - Displays "Patient not found"

This function uses a **linear search approach**

5.Update Patient Information

Description

- Allows modification of an existing patient's details.

Implementation Details

- Restricted to the **receptionist** • The system:
 - Searches for the patient by ID

- Stores the previous node reference ○

Removes the patient from the queue ●

Updated information is collected:

- New name
- New condition → new priority
- The updated patient is:
 - Reinserted using `enqueue()`
 - Positioned correctly based on new priority ●

The system ensures:

- Queue integrity is maintained
- Updated data is saved

6.Cancel Patient

Description

- Removes a patient from the queue.

Implementation Details

- Restricted to the **receptionist** ● The system:
 - Searches for the patient by ID ○

Maintains a pointer to the previous node ● Once found:

- Adjusts pointers to remove the node
- Frees memory
- Displays confirmation message
- Saves updated queue to file

7.Doctor Priority Override

Description

- Allows a doctor to change the priority of a patient.

Implementation Details

- Restricted to the **doctor role** ● The system:
 - Searches for the patient by ID
 - Displays current details ● The doctor selects a new priority (1–3) ● The system:
 - Removes the patient from the queue
 - Updates priority and condition
 - Reinserts using `enqueue()`
- This ensures:
 - Correct queue ordering after modification
- Changes are saved immediately **File**

Handling Mechanism

- ❖ The system includes file handling functionality to enable persistent storage of patient data. Patient records are stored in a text file named "`Patients.txt`" using a comma-separated format that includes ID, name, condition, priority, and arrival time.
- ❖ When saving data, the system writes all patient records from the queue into the file. If the queue is empty, the user is prompted to confirm whether they want to overwrite the file.
- ❖ When loading data, the system reads each line from the file, parses the values, and reconstructs the queue by inserting each patient using the enqueue operation. This ensures that the correct priority ordering is maintained.
- ❖ Additionally, the system prevents duplicate entries during loading and updates the internal arrival time counter to maintain consistency.

1.File Saving

Description

- Stores all patient data in a file for future retrieval. **Implementation**

Details

- Data is written to `Patients.txt`

- Each record is stored in a structured format:

ID,Name,Condition,Priority,ArrivalTime

- The system:

- Traverses the queue ◦

Writes each node to the file • If the

queue is empty:

- User confirmation is required before overwriting

2.File Loading

Description

➤ Restores patient data from file into the system.

Implementation Details

- The system:

- Clears the current queue

- Opens the file •

For each line:

- Parses data using string stream

- Reconstructs patient object •

Ensures:

- No duplicate IDs are inserted

- Arrival time is preserved

- Patients are inserted using `enqueue()` to maintain order

Memory Management

The system uses dynamic memory allocation for managing the linked list nodes.

- Nodes are created using `new`

- Memory is freed using `delete` during dequeue operations

This ensures efficient memory usage and prevents memory leaks.

Program Control Flow

The system operates through a **menu-driven loop**:

1. Login into the system
 2. Display menu options
 3. Accept user input
 4. Execute selected operation
 5. Repeat until user chooses to logout and exit
- Before exiting, the system prompts the user to save data, ensuring no information is lost unintentionally.

Error Handling and User Interaction

- Prevents invalid menu selections
- Checks id duplications and prevents them
- Handles file opening errors
- Confirms critical operations (like overwrite or exit)
- Provides clear feedback messages to the user

8.Sample Input and Output Demonstration

- ❖ This section demonstrates how the Hospital Queue Management System operates through example interactions. These examples illustrate the system's functionality and correctness.

Login page

```
--- Login ---
Username: admin
Password: 123
Receptionist login successful.

--- Hospital Queue System ---
1. Add Patient
2. Serve Patient
3. Display Patients
4. Search Patient
5. Save To File
(user settings)
```

Adding Patients Input:


```
8. cancel patient
9. Doctor Override
10. Logout
11. Exit
Enter choice: 1
Enter Patient ID: 1
Enter Patient Name: Kaleb Abebe
1. Severe Injury
2. Serious Illness
3. Mild Condition
3
ser settings)
```

Output:

```
10. Logout
11. Exit
Enter choice: 1
Enter Patient ID: 1
Enter Patient Name: Kaleb Abebe
1. Severe Injury
2. Serious Illness
3. Mild Condition
3
Patient added successfully.
Saved successfully.
(user settings)
```

Displaying Patients Output:

```
8. cancel patient
9. Doctor Override
10. Logout
11. Exit
Enter choice: 3

Patients in Queue:
ID: 3, Name: Germachew Abera, Condition: Serious Illness, Priority: 2
ID: 2, Name: Surafel Derege, Condition: Serious Illness, Priority: 2
ID: 1, Name: Kaleb Abebe, Condition: Mild Condition, Priority: 3

ser settings)
```

Serving a Patient Input:

```
9. Doctor Override
10. Logout
11. Exit
Enter choice: 2

Serving Patient:
ID: 3
Name: Germachew Abera
Condition: Serious Illness
Priority: 2
Saved successfully.

ser settings)
```

Searching for a Patient Output:

```
10. Logout
11. Exit
Enter choice: 4
Required P_ID: 2

Patient Found:
ID: 2
Name: Surafel Derege
Condition: Serious Illness
Priority: 2

ser settings)
```

Updating a Patient Input:

```
Enter Patient ID to update: 2

Current Details:
Name: Surafel Derege
Condition: Serious Illness
Priority: 2
Enter New Name: Surafel Dagnachew
1. Severe Injury
2. Serious Illness
3. Mild Condition
2
ser settings)
```

Output:

```
Name: Surafel Derege
Condition: Serious Illness
Priority: 2
Enter New Name: Surafel Dagnachew
1. Severe Injury
2. Serious Illness
3. Mild Condition
2
Patient updated successfully.
Saved successfully.
```

ser settings)

Saving Data

```
3. Display Patients
4. Search Patient
5. Save To File
6. Load From File
7. Update patient
8. cancel patient
9. Doctor Override
10. Logout
11. Exit
Enter choice: 5
Saved successfully.
```

ser settings)

Loading Data

```
6. Load From File
7. Update patient
8. cancel patient
9. Doctor Override
10. Logout
11. Exit
Enter choice: 6
Warning: Loading will discard current unsaved data. do you want to continue?
1
Data Loaded successfully.
```

ser settings)

Cancel patient Input&output

```
6. Load From File
7. Update patient
8. cancel patient
9. Doctor Override
10. Logout
11. Exit
Enter choice: 8
Enter Patient ID to cancel: 2
Patient cancelled successfully.
Saved successfully.
```

user settings)

Doctor override input&output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Enter choice: 9
Enter Patient ID for doctor override: 1
```

```
Current Details:
ID: 1
Name: Kaleb Abebe
Condition: Mild Condition
Priority: 3
```

```
Doctor Override - Set New Priority:
1. Severe Injury
```

user settings)

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Priority: 3

Doctor Override - Set New Priority:
1. Severe Injury
2. Serious Illness
3. Mild Condition
Doctor Override - Set New Priority:
2
Doctor override applied successfully.
Saved successfully.
```

user settings)

Logout Output

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
7. Update patient
8. cancel patient
9. Doctor Override
10. Logout
11. Exit
Enter choice: 10
Logging out...

--- Login ---
Username: doctor
Password: 456
user settings)
```

Exit Program Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
8. cancel patient
9. Doctor Override
10. Logout
11. Exit
Enter choice: 11
Do you want to save before exiting? (1=Yes, 0=No): 1
Saved successfully.
Exiting program.
[1] + Done
"/usr/bin/gdb" --inter
icrosoft-MIEngine-In-adink1zh.5yx" 1>"/tmp/Microsoft-MI
natnhaelgetachew@penguin: /bin/X11/X11/X11$
user settings)
```

9.Conclusion

- In conclusion, this project has effectively demonstrated the application of fundamental concepts from Data Structures and Algorithms in solving a real-world problem. The hospital queue management system was designed using an efficient priority-based approach, where patients are organized according to the severity of their condition and their arrival time.
- The implementation of a **priority queue which was implemented by using a single linked list** played a central role in optimizing the system, ensuring that patients with higher urgency are treated first while maintaining fairness. This highlights the importance of selecting appropriate data structures to improve performance and efficiency. Additionally, algorithmic thinking was applied in designing insertion, deletion, and sorting mechanisms to manage patient records effectively.
- Throughout the project, key concepts such as **time complexity, data organization, and algorithm efficiency** were considered, ensuring that the

system performs reliably even as the number of patients increases. The project also reinforced the understanding of how abstract data structures can be implemented and applied in practical scenarios.

- ❖ Overall, this work emphasizes the critical role of Data Structures and Algorithms in building efficient and scalable systems. It serves as a strong example of how theoretical concepts learned in class can be applied to develop practical, real-world solutions.

References

- Data Structures and Algorithms lecture materials.
- Fundamentals of programming I&II.
- C++ Standard Library Documentation